

Introduction

AI-Based Programming

Week 1: AI System Architecture & The Production Gap

Amr Eledkawy, Ph.D.

The "Notebook" vs. The "System"

Notebook Environment

- Static CSV
- Focus on Accuracy
- "Run All Cells"
- Manual Retraining

The "Notebook" vs. The "System"

Production System

- Streaming API / Live DB
- Focus on Latency & Reliability
- CI/CD, Unit Tests, Version Control
- Automated Pipelines

The "Notebook" vs. The "System"

Feature	Notebook Environment	Production System
Data Source	Static CSV (Manual)	Streaming API / Live DB
Scope	Focus on Accuracy	Focus on Latency & Reliability
Code	"Run All Cells"	Unit Tests, Version Control
Updates	Manual Retraining	Automated Pipelines / Online Learning

Hidden Technical Debt in ML Systems

ML Code is **less**

- Configuration Management
- Data Collection & Verification
- Resource Management (GPU/CPU)
- Serving Infrastructure

The 5-Stage AI Pipeline

- **Ingestion:** Batched (Historical) vs. Streaming (Live)
- **Features:** Scaling, Encoding, Validation
- **Model:** The inference engine
- **Reasoning:** Business logic, thresholds, safety
- **Service:** API response, UI update, actuation

The 5-Stage AI Pipeline



1. Ingestion

Batched (Historical) vs.
Streaming (Live)



2. Features

Scaling, Encoding,
Validation & Schema



3. Model

The inference engine
(The "Prediction")



4. Reasoning

Business logic,
thresholds, safety checks



5. Service

API response, UI update,
Robot actuation

Static vs. Dynamic Architectures

Offline/Batch

Train on past data, deploy for future

Safe, stable, slow to adapt

Online/Streaming

Continuous update with new samples

Use Cases: Stock trading, ad-click prediction

Microservices vs. Monoliths in AI

- **Decoupling:** Scale model (GPU) and web (CPU) independently
- **Polyglot:** Node.js web + Python model service
- **Trade-off:** Network latency vs. in-memory calls

The Feature Store (The Bridge)

Central place to define features

- **Problem:** Training/Serving code mismatch
- **Solution:** One source of truth

Data Collection Paradigms

- **Explicit:** User forms (High quality, Low volume)
- **Implicit:** Clicks, scrolls (Low quality, High volume)
- **APIs:** Clean, structured
- **Scraping:** Messy, fragile

The "Cold Start" Problem

How do you build AI with zero data?

- **Strategy 1:** Start with heuristics/rules
- **Strategy 2:** Synthetic data (simulation/LLMs)
- **Strategy 3:** Human-in-the-loop labeling

Feedback Loops – The System as Data Generator

Your software is a data-mining machine

- Model Predicts → User Reacts
- Reaction Logged → New Training Data
- Model Updates → The Flywheel

Course Roadmap

- **Phase 1 (Weeks 1-3):** Data & Architecture
- **Phase 2 (Weeks 4-7):** Advanced Logic & Models
- **Phase 3 (Weeks 8-9):** Perception & Reinforcement
- **Phase 4 (Weeks 10-12):** Deployment & Product

The Course Project

An End-to-End AI Application

- Automated Data Pipeline (Not static CSV)
- A functional Model (ML)
- A User Interface (Streamlit, Flask, React)

The Course Project



Requirement

End-to-End AI App
(Not just a static CSV)



Must Haves

1. Automated Pipeline
2. Functional Model
3. User Interface



Deliverable

GitHub Repo
Live Demo

Project Milestones

- **Week 4:** Proposal & Data Source
- **Week 8:** MVP System (Pipeline works)
- **Week 12:** Final Demo (Evaluation & UI)

Key Takeaways

- Architecture defines performance ceiling
- Think in Pipelines, not Scripts

Questions?

Open Floor for Discussion